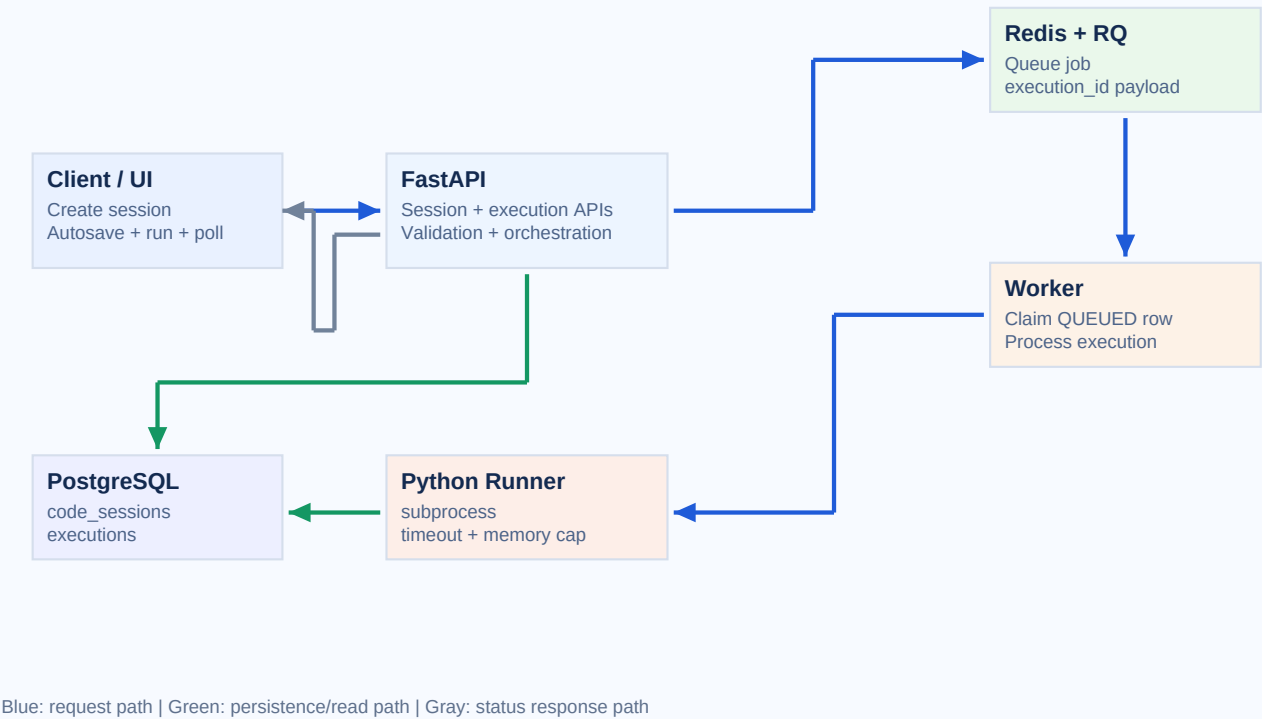


Live Code Execution & Management

Edtronaut SWE Intern Backend Take-home

Candidate Hoang Van Thien	Role SWE Intern - Backend	Focus Async execution + reliable lifecycle
---	---	--

Architecture Overview



Problem Context

Why this system is needed

- Edtronaut job simulations need runnable code, not only file upload.
- Interviewers need immediate output signals ('stdout', 'stderr', status, runtime).
- Long-running user code must not block API request handling.

Backend challenge

- Keep session state stable during frequent autosave.
- Execute code safely at MVP level with realistic guardrails.
- Provide traceable execution lifecycle for review and debugging.

Design Goals and Scope

Primary goals

- Non-blocking run API with queue-based execution.
- Reliable state transitions: 'QUEUED -> RUNNING -> COMPLETED/FAILED/TIMEOUT'.
- Honest MVP safety: timeout, memory cap (best effort), language allowlist.
- Simple setup for reviewers using Docker Compose.

Explicitly out of scope

- Production-grade sandbox isolation.
- Full auth, multi-tenant policy engine, global rate limiting platform.
- Multi-language execution in this MVP.

Project Architecture Overview

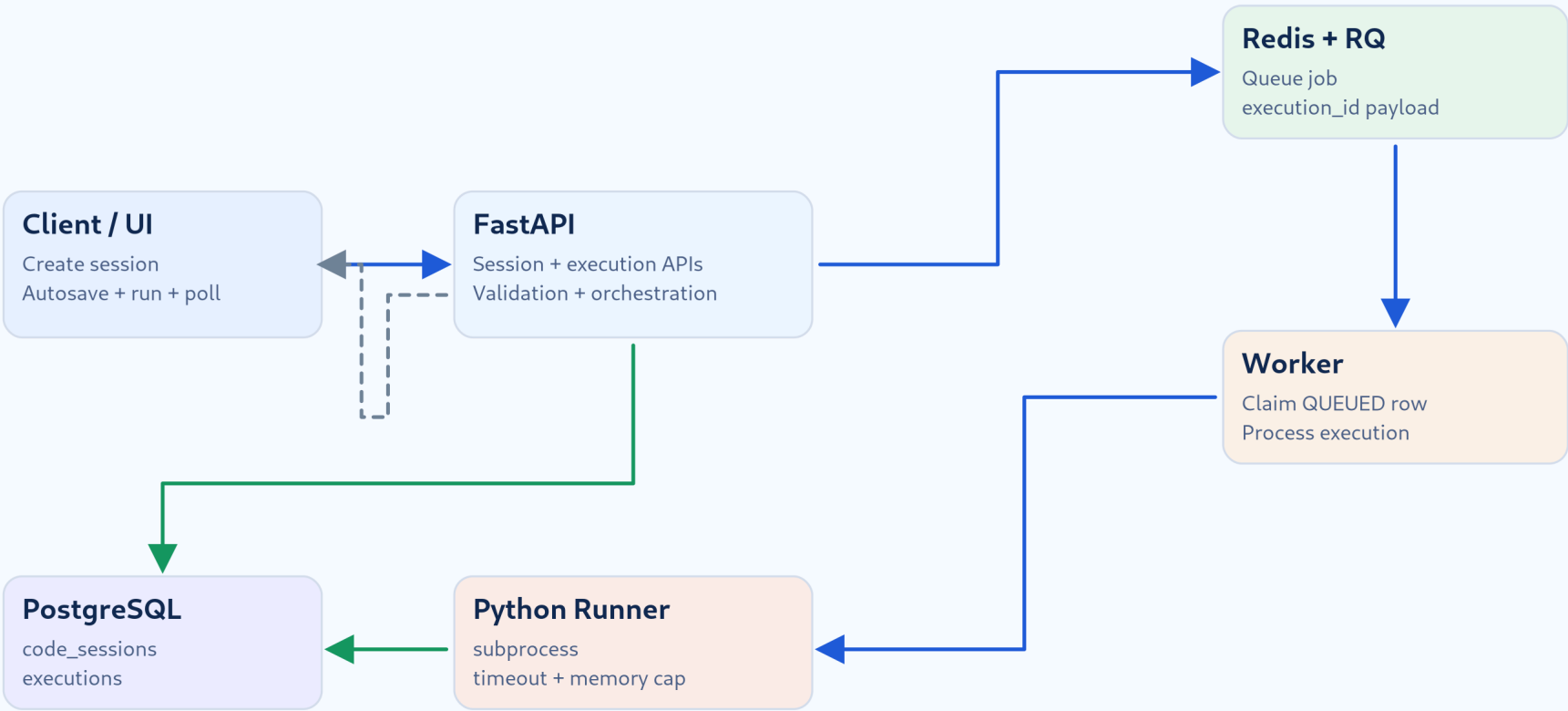
Implemented components

- FastAPI API service ('app/main.py', 'app/api/*')
- PostgreSQL persistence ('code_sessions', 'executions')
- Redis + RQ queue ('app/queue/rq.py')
- Background worker ('app/workers/rq_worker.py')
- Python subprocess runner ('app/runners/python_runner.py')

Key design intent

- API returns quickly after enqueueing.
- Worker handles execution and lifecycle persistence.
- Client polls execution result through read API.

Architecture Overview



Blue: request path | Green: persistence/read path | Gray: status response path

Core API Design

Session management APIs

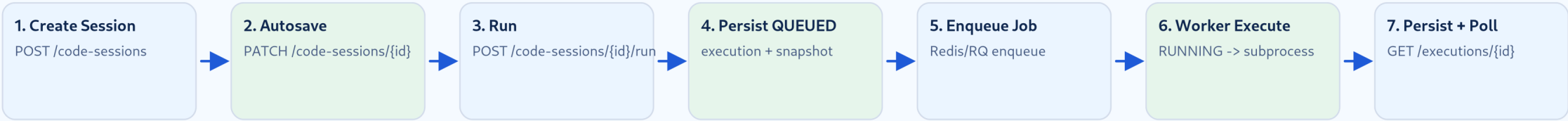
- 'POST /code-sessions': create ACTIVE session.
- 'PATCH /code-sessions/{session_id}': autosave source with version check.
- 'GET /code-sessions/{session_id}': reload existing session state.

Execution APIs

- 'POST /code-sessions/{session_id}/run'
- creates execution row first
- enqueues background job
- returns 'execution_id' + 'QUEUED' immediately ('202')
- 'GET /executions/{execution_id}'
- returns current status and stored output fields

End-to-End Request Flow

End-to-End Request Flow



Execution terminal states



Happy path is linear; terminal outcomes branch cleanly from RUNNING.

Main flow

- Create code session.
- Autosave source updates ('PATCH') while editing.
- Trigger run ('POST /run') with optional 'stdin_data'.
- API stores immutable 'source_code_snapshot' and returns quickly.
- Worker claims queued execution and runs subprocess.
- Client polls 'GET /executions/{id}' until terminal state.

Terminal states

- 'COMPLETED'
- 'FAILED'
- 'TIMEOUT'

Data Model and State Management

Data Model (code_sessions -> executions)



State model: QUEUED -> RUNNING -> COMPLETED / FAILED / TIMEOUT

‘code_sessions‘

- ‘id‘, ‘language‘, ‘template_code‘, ‘current_source_code‘
- ‘status‘ (ACTIVE), ‘version‘
- ‘created_at‘, ‘updated_at‘

‘executions‘

- ‘id‘, ‘session_id‘, ‘status‘
- ‘source_code_snapshot‘, ‘stdout‘, ‘stderr‘, ‘error_message‘
- ‘execution_time_ms‘, ‘retry_count‘
- ‘queued_at‘, ‘started_at‘, ‘finished_at‘

State behavior

- Snapshot is immutable per run for reproducibility.
- Worker updates only rows currently in ‘QUEUED‘ to avoid duplicate processing.

Execution Worker and Isolation Strategy

Worker processing ('process_execution')

- Atomic claim from 'QUEUED' to 'RUNNING'.
- Execute Python code via subprocess in temporary directory.
- Persist terminal state + outputs + timing.

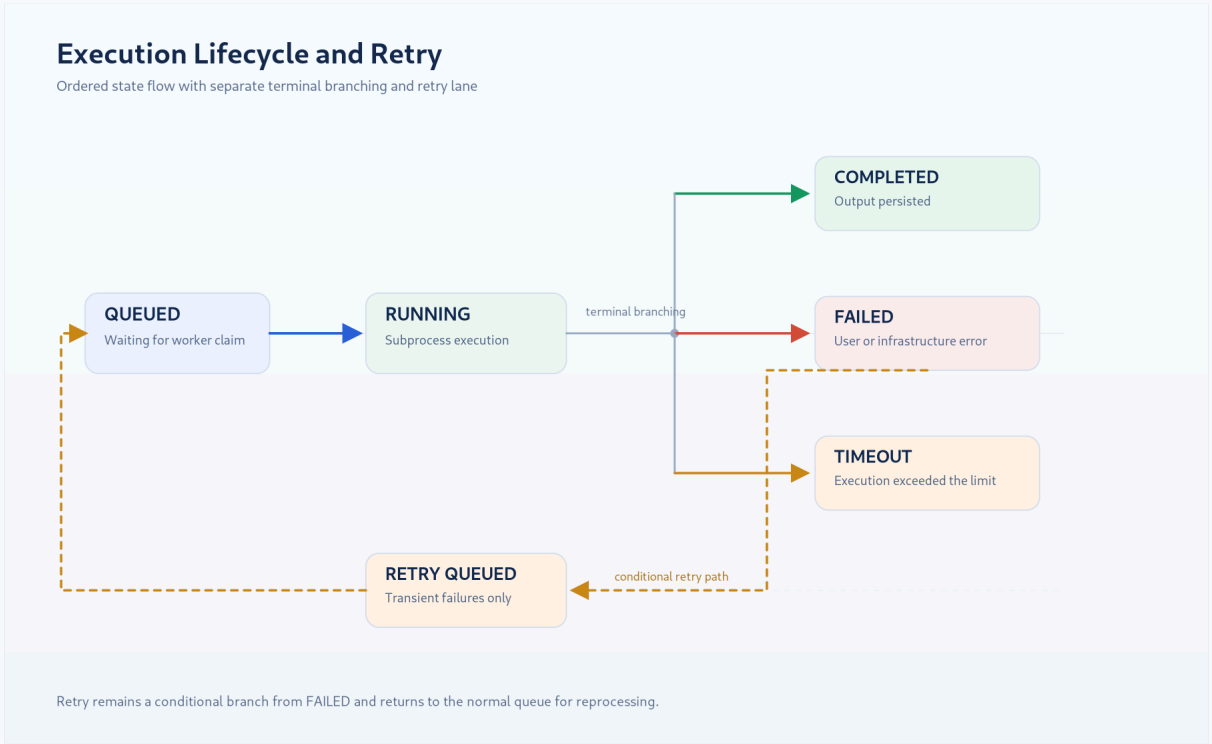
MVP isolation strategy

- Time limit via subprocess timeout.
- Memory cap via 'RLIMIT_AS' on POSIX (best effort).
- Python-only allowlist for execution requests.

Important caveat

- This is MVP isolation, not a hardened multi-tenant sandbox.

Reliability, Idempotency, and Failure Handling



Reliability controls implemented

- Idempotency guard: duplicate jobs skip non-‘QUEUED’ rows.
- Queue enqueue failure after DB commit is patched:
- execution is updated to ‘FAILED’
- clear error message is stored
- API returns ‘503’

Retry policy

- Retry only for transient runner infrastructure errors.
- Retry cap controlled by ‘EXECUTION_INFRA_MAX_RETRIES’.
- User-code errors and timeout are not retried by default.

Safety and Abuse Protection

Safety controls implemented

- Language allowlist ('python').
- Source size limit ('EXECUTION_MAX_SOURCE_SIZE_BYTES').
- Min run interval per session ('EXECUTION_MIN_INTERVAL_SECONDS').
- Timeout and best-effort memory limits in runner.

Observability

- Execution lifecycle logging with structured key fields.
- Stored timestamps for queued, started, finished stages.

Remaining risk

- No advanced tenant isolation or system-wide abuse control layer yet.

Scalability Considerations

What scales now

- API layer is stateless and horizontally scalable.
- Worker can scale by adding more consumers on the same Redis queue.
- Queue decouples request latency from execution latency.

Expected bottlenecks

- Single Redis instance and single PostgreSQL write path.
- Polling frequency load on read endpoints under heavy concurrency.

Practical next steps

- Add queue backlog metrics + autoscaling triggers.
- Add dead-letter queue and richer retry backoff.
- Introduce event-driven updates when needed.

Technology Choices and Trade-offs

Why this stack

- FastAPI: fast iteration, typed contracts.
- SQLAlchemy + PostgreSQL: explicit persistence and query control.
- Redis + RQ: simple async job model for MVP.
- Docker Compose: reproducible local review environment.

Trade-offs accepted

- Simplicity and clarity over full production hardening.
- Polling over WebSocket push for lower complexity.
- 'create_all' bootstrap over full migration pipeline in MVP.

Repository Deliverables

Delivered artifacts

- Structured backend modules: API, services, models, queue, worker, runner.
- 'README.md' with setup, environment variables, API usage, and checks.
- 'DESIGN.md' with architecture, flow, lifecycle, reliability, and trade-offs.
- 'Dockerfile' + 'docker-compose.yml' for one-command startup intent.
- Test suite for APIs, policy logic, runner behavior, and execution service.

Local verification commands

- 'python -m unittest discover -s tests -v'
- 'docker compose config --quiet'
- 'docker compose up --build'

Gaps and Future Improvements

Current limitations

- Sandbox isolation is not production-grade.
- No auth, RBAC, or tenant-level quotas.
- No full observability stack (metrics/tracing/log shipping).
- No migration tool chain in this MVP.

Future improvements

- Container/VM isolation per execution.
- Dead-letter queue + backoff strategy.
- Rich execution telemetry and alerting.
- Multi-language execution support.

Conclusion

Backend assessment

- The project implements the required MVP backend workflow end-to-end.
- Async execution lifecycle is explicit, traceable, and test-covered.
- Reliability and safety controls are practical for intern-level scope.

Submission position

- Strong backend-first take-home submission.
- Honest about production gaps and next steps.